Hoare Logic

LN chapter 5, 6, 9.3

Hoare Logic is used to reason about the correctness of programs. In the end, it reduces a program and its specification to a set of verifications conditions.

Overview

- Part I
  - Hoare triple
  - Rules for basic statements // SEQ, IF, ASG
  - Weakest pre-condition
  - Example
- Part II : loop
- Handling few more basic constructs
  - array
  - specification at the program level

Part 1 : reasoning about basic statements

Hoare triple

- A “Hoare triple” is a simple way to specify the relation between a program’s input and end result. Example:

$$\underbrace{\{*\ y \neq 0\ *\}}_{\text{pre-condition}} \quad \underbrace{Pr(x,y)}_{\text{the program being specified}} \quad \underbrace{\{*\ return = x/y\ *\}}_{\text{post-condition}}$$

- Meaning: if the program is executed on a state satisfying the pre-condition, it will terminate in a state satisfying the post-condition. (so-called “total correctness interpretation”).

Partial and total correctness interpretation

$$\{* \; P \; *\} \quad Pr(x) \quad \{* \; Q \; *\}$$

- **Total correctness interpretation** of Hoare triple: if the program is executed on a state satisfying P, it will **terminate** in a state satisfying Q.
- **Partial correctness interpretation**: if the program is executed on a state satisfying P, and if it terminates, it will terminate in a state satisfying Q.
- Partial correctness is of course less complete, but on the other hand easier to prove. Useful in situations where we can afford to postpone concerns about termination.

Difficult to capture with standard Hoare triples

- We have to note that Hoare triples have limitation. Not every type of behavioral properties can be (or can naturally be) captured with Hoare triples. Examples:
  - Certain actions within a program must occur in a certain order. An alternative formalism to express this: CSP
  - A certain state in the program must be visited infinitely often. Alternative formalism: temporal logic.
  - A certain goal must be reached despite the presence of adversaries in the program's environment that may try to fake information. Alternative formalism: logic of belief.

Hoare Logic

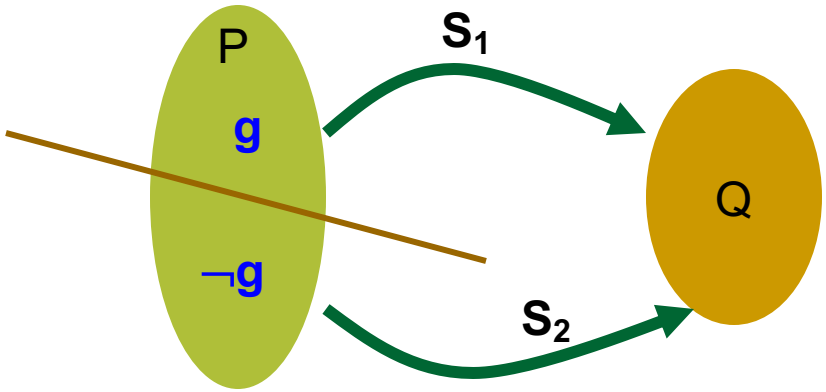
Hoare Logic is a logic to prove the correctness of a program in terms of the validity of Hoare triples. It provides a set of inference rules like the one below, to ”decompose” statement sequence (SEQ Rule):

$$\frac{\{*\ P\ *\}\ \ S_1\ \ \{*\ Q\ *\}\ \ ,\ \ \{*\ Q\ *\}\ \ S_2\ \ \{*\ R\ *\}}{\{*\ P\ *\}\ \ S_1\ ;\ S_2\ \ \{*\ R\ *\}}$$

Inference rule for IF

$$\frac{\{*\ P \wedge g\ *\}\ \ S_1\ \ \{*\ Q\ *\}\ \ ,\ \ \{*\ P \wedge \neg g\ *\}\ \ S_2\ \ \{*\ Q\ *\}}{\{*\ P\ *\}\ \textbf{if}\ \ g\ \ \textbf{then}\ \ S_1\ \ \textbf{else}\ \ S_2\ \ \{*\ Q\ *\}}$$

$\{*\ P \wedge g\ *\}\ \ S_1\ \ \{*\ Q\ *\}$



$\{*\ P \wedge \neg g\ *\}\ \ S_2\ \ \{*\ Q\ *\}$

Inference rule for assignment, attemp-1

$$\frac{??}{\{*\ P\ *\}\quad \mathbf{x{:=}e}\quad \{*\ Q\ *\}}$$

- **Idea: let's** first construct a pre-condition *W*, that holds before the assignment, **if and only if** Q holds after the assignment.
- Then we can equivalently prove $P \Rightarrow W$

Examples

- {* W? *} **x:=10** {* x=y *}

  W: 10=y

- {* W? *} **x:=x+a** {* x=y *}

  W: x+a = y

- More generally, such a *W* can be obtained by applying this substitution: $Q[e/x]$

Assignment

- Theorem:

  Q holds after x:=e iff Q[e/x] holds before the assignment.

- Which leads to the following "proof rule" :

$$\{*\ P\ *\}\ \ x{:=}e\ \ \{*\ Q\ *\} \quad = \quad P \Rightarrow Q[e/x]$$

Few other basic rules

- **Pre-condition strengthening:**

$$\frac{\vdash P \Rightarrow Q \quad , \quad \{*\ Q\ *\}\ \ S\ \ \{*\ R\ *\}}{\{*\ P\ *\}\ \ S\ \ \{*\ R\ *\}}$$

- **Post-condition weakening:**

$$\frac{\{*\ P\ *\}\ \ S\ \ \{*\ Q\ *\}\ , \quad \vdash Q \Rightarrow R}{\{*\ P\ *\}\ \ S\ \ \{*\ R\ *\}}$$

Few other basic rules

- **Conjunction of Hoare triples:**

$$\frac{\{*\ P_1\ *\}\ \ S\ \ \{*\ Q_1\ *\}\ \ ,\ \{*\ P_2\ *\}\ \ S\ \ \{*\ Q_2\ *\}}{\{*\ P_1 \wedge P_2\ *\}\ \ S\ \ \{*\ Q_1 \wedge Q_2\ *\}}$$

- **Disjunction of Hoare triples**

$$\frac{\{*\ P_1\ *\}\ \ S\ \ \{*\ Q_1\ *\}\ \ ,\ \{*\ P_2\ *\}\ \ S\ \ \{*\ Q_2\ *\}}{\{*\ P_1 \vee P_2\ *\}\ \ S\ \ \{*\ Q_1 \vee Q_2\ *\}}$$

How does a proof proceed now ?

- {* x≠y *} tmp:= x ; x:=y ; y:=tmp {* x≠y *}

- Rule for SEQ requires you to come up with intermediate assertions:

  {* x≠y *} tmp:= x {* ? *} ; x:=y {* ? *} ; y:=tmp {* x≠y *}

- If we manage to come up with the intermediate assertions, Hoare logic's rules tell us how to prove the claim. However, the rules **do not** tell us how to come up with these intermediate assertions in the first place,

Weakest pre-condition (wp), LN Sec. 6.2

- Imagine we have this function:

$$\textbf{wp} : \text{Stmt} \rightarrow \text{Pred} \rightarrow \text{Pred}$$

  such that **wp** *S Q* gives a ***valid*** pre-cond. That is, executing *S* in any state in this pre-cond results in a state in *Q*. Again, we can have two variations of such a function:
  - Partial correctness based wp assumes *S* to always terminates.
  - Total correctness wp → the produced pre-condition guarantees that *S* terminates when executed on that pre-condition.
- Even better: let **wp** construct the ***weakest*** *valid* pre-cond.

Weaker and stronger

- If p $\Rightarrow$ q is *valid*, we say that *p* is stronger than *q*
- Conversely*, q* is weaker than *p*.
- A state-predicate *p* can also be seen as characterizing a set of states, namely those states on which *p* is true.
- "*p* is stronger than *q*" is the same as saying, in terms of set, that "*p* is subset of *q*".

Weakest pre-condition

- We can "characterize" **wp**, as follows (Def. 6.2.3):

$$\{*\ P\ *\}\ \ S\ \ \{*\ Q\ \ *\} \quad \equiv \quad P \Rightarrow \mathbf{wp}\ S\ Q$$

- *Corollary-1*: Such wp always produces a valid pre-cond. You can prove:

$$\{*\ \ \mathbf{wp}\ S\ Q\ *\} \quad S \quad \{*\ Q\ *\}$$

- *Corollary-2* : **wp** reduces program verification problems to proving implications, for which we already have a "tool" for (Predicate Logic).

Weakest pre-condition

- Again, the characterization of wp:

$$\{^* \; P \; ^*\} \;\; S \;\; \{^* \; Q \; ^*\} \quad \equiv \quad P \Rightarrow \mathbf{wp} \; S \; Q$$

- *Corollary-3*: the reduction is complete. That is, if the implication above is not valid, so is the specification.

  Also interesting: a counter example demonstrating the invalidity of the implication essentially describes an input for the program/statement S, that leads to a final state that is not in Q. Such an input is useful for debugging S.

But...

- But this characterization is **not constructive**:

$$\{^* P \,^*\} \; S \; \{^* Q \; ^*\} \quad \equiv \quad P \Rightarrow \mathbf{wp}\; S\; Q$$

- That is, it does not tell us how to actually calculate this weakest pre-condition that wp is supposed to produce.
- To be actually usable, we need to come up with a constructive definition for wp.

Some notes before we attempt to define wp

- This is the meta property we want to have (I swap the places, but that does not matter):

$$P \Rightarrow \textbf{wp}\ S\ Q \quad \equiv \quad \{*\ P\ *\}\ S\ \{*\ Q\ *\}$$

- A definition/implementation of wp should be at least "**sound**"; it should produce a safe pre-condition:

$$P \Rightarrow \textbf{wp}\ S\ Q \quad \Rightarrow \quad \{*\ P\ *\}\ S\ \{*\ Q\ *\}$$

- A wp definition that satisfies the reverse property is called "**complete**". I formulate it in contraposition to emphasize that it means, if the implication is invalid, the Hoare triple is also not valid:

$$P \not\Rightarrow \textbf{wp}\ S\ Q \quad \Rightarrow \quad \cancel{\{*\ P\ *\}\ S\ \{*\ Q\ *\}}$$

Weakest pre-condition of skip and assignment

$$\textbf{wp}\ \ \textbf{skip}\ \ Q \quad = \quad Q$$

$$\textbf{wp}\ \ \textbf{(x:=e)}\ \ Q \quad = \quad Q[e/x]$$

Are those definitions "good" ?

- That is, do they uphold the wp characterization we discussed before?
- For "skip" it is clear.
- For assignment, here is the proof (simple) :

$\{P\}$ **x:=e** $\{Q\}$

= { Hoare logic rule for assignment}

$P \Rightarrow Q[e/x]$

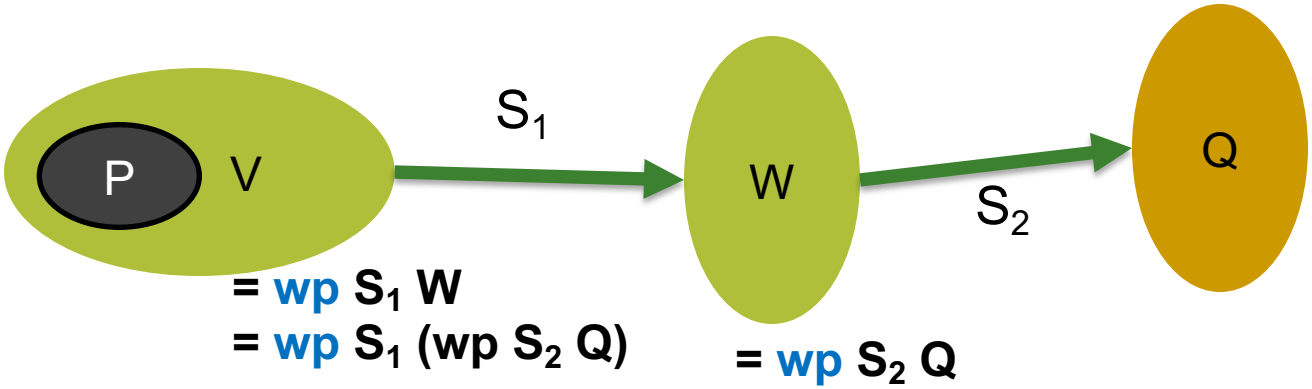
= { def. wp on assignment }

$P \Rightarrow$ **wp** (**x:=e**) Q

wp of SEQ

$$\textbf{wp}\ (S_1\ ;\ S_2)\ \ Q\ \ =\ \ \textbf{wp}\ \ S_1\ \ (\textbf{wp}\ \ S_2\ \ Q)$$

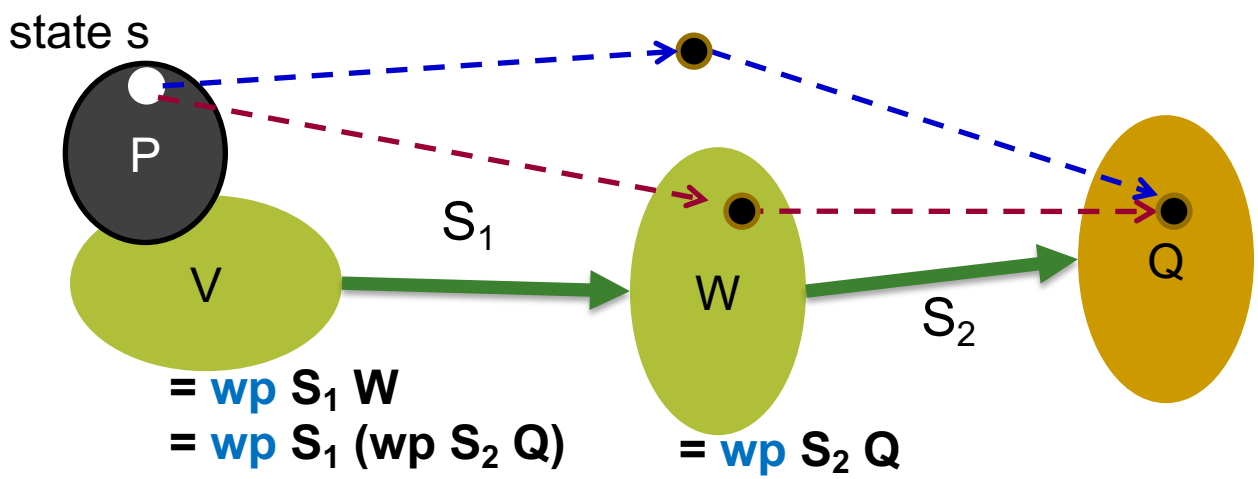
Proving that this def. of wp upholds the wp-characterization is more complicated now and involves an inductive argument. We will only give a sketch here. The first part of the argument is that any pre-cond P that implies V below will also satisfies {P} S_1;S_2 {Q}.



wp of SEQ

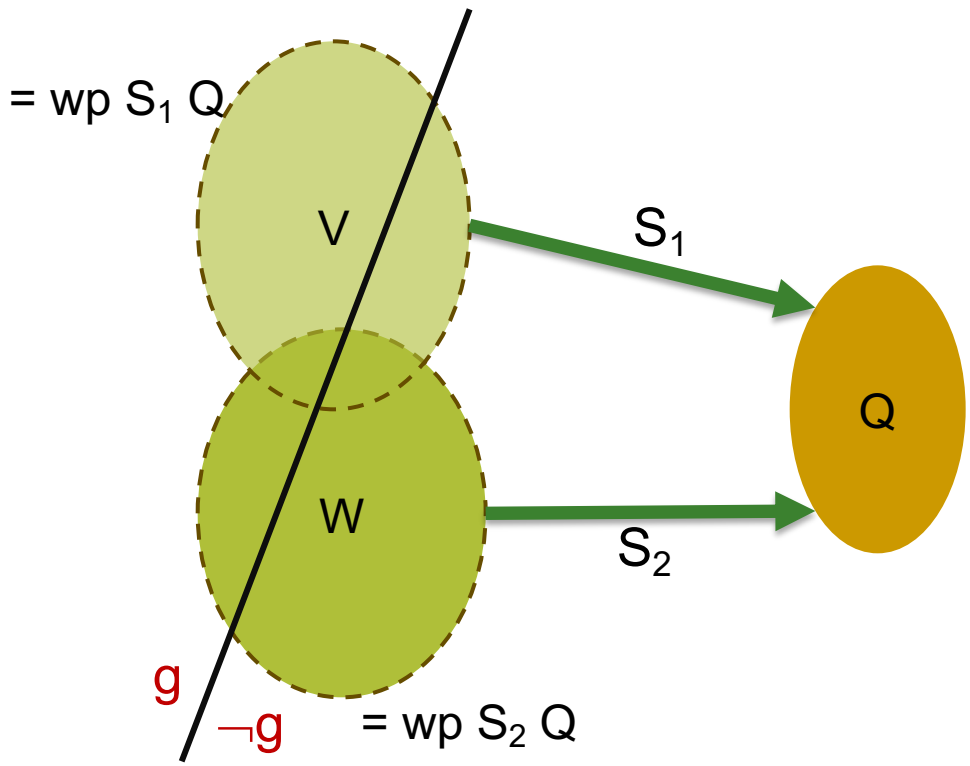
$$\textbf{wp}\ (S_1\ ;\ S_2)\ \ Q\ \ =\ \ \textbf{wp}\ S_1\ (\textbf{wp}\ S_2\ Q)$$

The second part of the argument is to show that if {P} S_1;S_2 {Q} holds, then P implies V below. We argue this by contradiction. Suppose there is some state s satisfying P, but not V (see below). Since {P} S_1;S_2 {Q}, executing S_1;S_2 on s will bring us to Q. But this leads to contradictions depicted below.



wp of IF

wp (**if** g **then** S_1 **else** S_2) Q = $(g \wedge$ **wp** $S_1\ Q) \vee (\neg g \wedge$ **wp** $S_2\ Q)$



Alternative formulation of wp IF

$$\textbf{wp}\ (\textbf{if}\ g\ \textbf{then}\ S_1\ \textbf{else}\ S_2)\ Q \quad = \quad (g \wedge \textbf{wp}\ S_1\ Q) \vee (\neg g \wedge \textbf{wp}\ S_2\ Q)$$

$$=$$

$$(g \Rightarrow \textbf{wp}\ S_1\ Q) \wedge (\neg g \Rightarrow \textbf{wp}\ S_2\ Q)$$

This formulation for the **wp** of if-then-else is more convenient for working out proofs. E.g. in a deductive proof its conjunctive form would translate to proving two goals.

How does a proof proceed now ?

- {* $x \neq y$ *} tmp:= x ; x:=y ; y:=tmp {* $x \neq y$ *}

1. Calculate:

$$W \quad = \quad wp \; \boxed{(tmp := x \; ; \; x := y \; ; \; y := tmp)} \; \boxed{x \neq y}$$

2. Then prove: $x \neq y \Rightarrow W$

- We <u>calculate</u> the intermediate assertions, rather than manually figuring them out!

Example

{* $0 \le i \ \wedge \neg found \wedge \ (found \ = \ (\exists k : 0 \le k < i : \ a[k]=x))$ *}

$\Downarrow$

found := a[i]=x ;

$(a[i]=x) \ = \ (\exists k : 0 \le k < i+1 : \ a[k]=x)$

i:=i+1

$found \ = \ (\exists k : 0 \le k < i+1 : \ a[k]=x)$

{* $found \ = \ (\exists k : 0 \le k < i : \ a[k]=x)$ *}

$wp \ (x:=e) \ Q \ = \ Q[e/x]$

Some notes about the verification approach

- In our training, we prove $P \Rightarrow$ W by hands.
- In practice, to some degree this can be automatically checked using e.g. a SAT solver. With a SAT solver we instead check if:

  $\neg(P \Rightarrow W)$ is satisfiable

  If it is, then $P \Rightarrow W$ is **not** valid. Otherwise $P \Rightarrow W$ is valid.

- If the definition of wp is also ”complete” (in addition to being “sound”), the witness of the satisfiability of $\neg(P \Rightarrow W)$ is essentially an input that will expose a bug in the program → useful for debugging.

Part II : reasoning about loops

LN Section 6.3 -- 6.7, and 9.3

How to prove this ?

{* P *} **while** g **do** S {* Q *}

Unfortunately calculating **wp** won't work here. Essentially because we do not know upfront how many times the loop will iterate. We might still know that it will iterate for example N times, where N is a parameter of the program, but recall that calculating the wp of SEQ requires us to know concretely how many statements are being composed in the sequence.

Idea

Come up with a predicate ***I***, so-called "**invariant**", that holds at the end of every iteration.

$iter_1$: // g // ; **S** {* **I** *}
$iter_2$: // g // ; **S** {* **I** *}
…
$iter_n$: // g // ; **S** {* **I** *} // last iteration
exit : // ¬g //

- Observation: it follows that I /\ ¬g must hold when the loop terminates (after the last iteration). The post-condition Q can be established if I /\ ¬g implies Q.

Ok, what kind of predicate is I ??

- We need an I that holds at the end of every iteration. Inductively, this means that we can assume it to hold at the start of the iteration (established by the previous iteration).

- So, it is sufficient to have an I satisfying this property:

$$\{* \; I \wedge g \; *\} \quad \mathbf{S} \quad \{* \; I \; *\}$$

Establishing the base case

The previous inductive argument does not however apply for the first iteration, because it has no previous iteration. Let's just insist that this first iteration should also maintain the property:

$$\{* \; I \wedge g \; *\} \quad \mathbf{S} \quad \{* \; I \; *\}.$$

Something else is now needed to guarantee that the first iteration can indeed assume I as its pre-condition. We can establish this by requiring that the original precondition P implies this I.

To Summarize

- Capture this in an inference rule (Rule 6.3.2):

$$
\begin{array}{ll}
P \Rightarrow I & \text{// setting up I} \\
\{* \; g \wedge I \; *\} \quad S \quad \{* \; I \; *\} & \text{// invariance} \\
I \wedge \neg g \Rightarrow Q & \text{// exit cond} \\
\hline
\{* \; P \; *\} \quad \textbf{while} \; g \; \textbf{do} \quad S \quad \{* \; Q \; *\} &
\end{array}
$$

- This rule is only good for **partial correctness** though.

Few things to note

- A special instance the previous rule is this (by taking I itself as P):

$$\frac{\{*\ g \wedge I\ *\}\quad S\quad \{*\ I\ *\} \qquad I \wedge \neg g \Rightarrow Q}{\{*\ I\ *\}\quad \textbf{while}\ g\ \textbf{do}\ S\quad \{*\ Q\ *\}}$$

- If I satisfies the above two conditions, we can extend an implementation of **wp** to take this I as the **wp** of the loop over Q as the post-condition.
- This allows you to chain I into the rest of **wp** calculation.
- Such a modified **wp** function would still be sound, though it is no longer complete (in other words, the resulting pre-condition may not be the weakest one).

Examples

- Prove the correctness of the following loops (partial correctness) :

{* true *} **while** $i \neq n$ **do** i++ {* i=n *}

{* $i=0 \wedge n=10$ *} **while** $i<n$ **do** i++ {* i=n *}

{* $i=0 \wedge s=0 \wedge n=10$ *} **while** $i<n$ **do** { s = s+2 ; i++ } {* s=20 *}

Proving termination

- Again, consider:

  {* P *} <u>while</u> g <u>do</u> S {* Q *}

- Idea: come up with an integer expression m, called "**termination metric**", satisfying :
  1. At the start of every iteration m > 0
  2. Each iteration decreases m

- Since m has a lower bound (condition 1), it follows that the loop cannot iterate forever. In other words, it will terminate.

Capturing the termination conditions

- At the start of every iteration $m > 0$:
  - $g \Rightarrow m \geq 0$
  - If you have an invariant: $I \wedge g \Rightarrow m > 0$
- Each iteration decreases m :

$$\{* \; I \wedge g \; *\} \quad C := m; \; S \quad \{* \; m < C \; *\}$$

To Summarize

- Rule B.1.8:

$P \Rightarrow I$ // setting up I

$\{* \; g \wedge I \; *\} \quad S \quad \{* \; I \; *\}$ // invariance

$I \wedge \neg g \Rightarrow Q$ // exit cond

$\{* \; I \wedge g \; *\} \quad C{:=}m;\ S \quad \{* \; m<C \; *\}$ // m decreasing

$I \wedge g \Rightarrow m > 0$ // m bounded below

$\{* \; P \; *\}$ **while** g **do** S $\{* \; Q \; *\}$

Alternatively it can be formulated as follows

- Rule B.1.7

$$
\begin{array}{ll}
\{*\ g \wedge I\ *\}\ \ S\ \ \{*\ I\ *\} & \text{// invariance} \\
I \wedge \neg g \Rightarrow Q & \text{// exit cond} \\
\{*\ I \wedge g\ *\}\ \ C{:=}m;\ S\ \ \{*\ m<C\ *\} & \text{// m decreasing} \\
I \wedge g \Rightarrow m > 0 & \text{// m bounded below} \\
\hline
\{*\ I\ *\}\ \ \underline{\text{while}}\ g\ \underline{\text{do}}\ \ S\ \ \{*\ Q\ *\} &
\end{array}
$$

- This is a special instance of B.1.7. On the other hand, together with the pre-condition strengthening rule it also implies B.1.7.

Examples

- Prove that the following programs terminate:

```
{* i≤n *}  while  i≠n  do  i++    {* true *}
```

```
{*  candy=100 ∧ apple=100 }

while candy>0 ∨ apple>0  do {
    if candy>0 then { candy--  ; apple += 2 }
    else  { apple-- }
}
                    {* true *}
```

A bigger example

- Let's now consider a less trivial example. The following program claims to check if the array segment a[0..n) consists of only 0's. Prove it :

```
{* 0≤n *}
   i := 0 ; r := true ;
  while  i<n  do  {  r := r ∧ (a[i]=0)   ;   i++ }
{*  r  = (∀k : 0≤k<n : a[k]=0) *}
```

- You will need to propose an invariant and a termination metric.
- This time, the proof will be quite involved. The rule to handle loops also generates multiple conditions.

Invariant and termination metric

```
{* 0≤n *}
  i := 0 ; r := true ;
  while  i<n  do  {  r := r ∧ (a[i]=0)   ;   i++ }
{*  r  = (∀k : 0≤k<n : a[k]=0) *}
```

Part-2 (no 1,2,3)

- Let's go with this choice of invariant I :

$$\underbrace{(r = (\forall k : 0 \le k < i : a[k]=0))}_{I_1} \wedge \underbrace{0 \le i \le n}_{I_2}$$

- The choice for the termination metric m is in this case quite obvious, namely: n - i

It comes down to proving these

1. (Exit Condition) $I \wedge \neg g \Rightarrow Q$

2. (Initialization Condition)

   {* given-pre-cond *} i := 0 ; r := **true** {* I *},

3. (Invariance) {* $I \wedge g$ *} body {* I *}

   Or equivalently, prove: $I \wedge g \Rightarrow$ **wp** body I

4. (Termination Condition) {* $I \wedge g$ *} (C:=m ; body) {* m<C *}
5. (Termination Condition) $I \wedge g \Rightarrow m>0$

Reformulating them in terms of wp

If the composing statements has no loops, we can convert the Hoare triples into implications towards wp. Hence reducing the problem to pure predicate logic problem:

1. (Exit Condition) $I \wedge \neg g \Rightarrow Q$
2. (Initialization Condition)
   given-pre-cond $\Rightarrow$ **wp** (i := 0 ; r := **true**) I
3. (Invariance) $I \wedge g \Rightarrow$ **wp** body I
4. (Termination Condition) $I \wedge g \Rightarrow$ **wp** (C:=m ; body) (m<C)
5. (Termination Condition) $I \wedge g \Rightarrow m>0$

Now we can use the previous proof system for predicate logic to prove those implications.

Top level structure of the proofs

- **PROOF** PEC (proof of the exit condition)
  [A1] $r = (\forall k : 0 \le k < i : a[k]=0)$
  [A2] $0 \le i \le n$
  [A3] $i \ge n$
  [G] $r = (\forall k : 0 \le k < n : a[k]=0)$
  .... (the proof itself)
  END

- **PROOF** PInit (proof of the initialization condition)
  [A1] $0 \le n$
  [G] $(true = (\forall k : 0 \le k < 0 : a[k]=0)) \wedge 0 \le 0 \le n$ // calculated wp
  ...
  END

Proof of init

PROOF PInit
[A1] $0 \le n$
[G] $(true = (\forall k : 0 \le k < 0 : a[k]=0)) \wedge 0 \le 0 \le n$ ← calculated wp
BEGIN

1. { follows from A1 } $0 \le 0 \le n$
2. { see subproof } $true = (\forall k : 0 \le k < 0 : a[k]=0)$

EQUATIONAL PROOF
$(\forall k : 0 \le k < 0 : a[k]=0)$
= { the domain is false }
$(\forall k : false : a[k]=0))$
= {$\forall$ over empty domain }
true
END

3. { conjunction of 1 and 2 } G

end

Proof of I's invariance

PROOF PIC (proof of the invariance condition)

[A1] $r = (\forall k : 0 \le k < i : a[k]=0)$

[A2] $0 \le i \le n$

[A3] $i < n$

[G1] $r \wedge (a[i]=0) = (\forall k : 0 \le k < i+1 : a[k]=0)$

[G2] $0 \le i+1 \le n$

BEGIN

1. { see the subproof below } G1

EQUATIONAL PROOF

$(\forall k : 0 \le k < i+1 : a[k]=0)$

= { dom. merge , PIC.A2 }

$(\forall k : 0 \le k < i \vee k=i : a[k]=0)$

= { $\forall$ domain-split , justified by $0 \le i$ (PIC.A2) }

$(\forall k : 0 \le k < i : a[k]=0) \wedge (\forall k : k=i : a[k]=0)$

= { PIC.A1 }

$r \wedge (\forall k : k=i : a[k]=0)$

= { quant. over singleton }

$r \wedge (a[i]=0)$

END

2. { A2 implies $0 \le i+1$ and A3 implies $i+1 \le n$ } G2

END

I_1

I_2

Loop guard

caculated from wp

Top level structure of the proofs of termination

- **PROOF** TC1 (proof of the 1st termination condition)
  [A1] r = (∀k : 0≤k<i : a[k]=0)
  [A2] 0≤i≤n
  [A3] i<n
  [G] n - (i+1) < n-1 // calculated wp
  ....
  END

- **PROOF** TC2
  [A1] r = (∀k : 0≤k<i : a[k]=0)
  [A2] 0≤i≤n
  [A3] i<n
  [G] n - i > 0
  ....
  END

Loop with breaks, LN Sec. 9.3

- There is no “break” statement in uPL, but we can still look at the concept behind it, namely to stop a loop earlier because we know that it has achieved its objective.

```
i := 0 ; r := true ;
while  i<n do  {
    r := r ∧ (a[i]=0)  ;
    i++
}
```

```
i := 0 ; r := true ;
while  i<n ∧ ¬r  do  {
    r := r ∧ (a[i]=0)  ;
    i++
}
```

- How to prove that breaking the loop early is indeed safe?

Loop with breaks

- Consider the following loop, which has been proven correct using an **invariant I** and a **termination metric m**.

(1) {* P *} **while** g **do** S {* Q *}

- Suppose now we optimize the loop by adding a ”break” to let it terminate early in some situation:

(2) {* P *} **while** g $\wedge$ **h** **do** S {* Q *}

- The validity of (1) implies:
  - (2) will also terminate
  - (2) will also maintain I’s invariance
  - If g becomes false, (2) will terminate in Q

Loop with breaks

(2) $\{*\ P\ *\}$ **while** $g \wedge h$ **do** S $\{*\ Q\ *\}$

So to prove that (2) is valid, given (1) from the previous slide, we only need to prove that when (2) terminates early because h becomes false, then it will also terminate in Q. In other words, we only need to prove:

$$I \wedge g \wedge \neg h \Rightarrow Q$$

where I is the invariant used in (1).

Example

```
{* 0≤n *}

i := 0 ; r := true ;
while  i<n  ∧ r  do  {
    r := r ∧ (a[i]=0)  ;
    i++
}

{*  r  = (∀k : 0≤k<n : a[k]=0) *}
```

We have proven the basic form of this program (without the break with ∧ r) before. Now prove that with the break the program is still correct.

Dealing with more constructs

LN Section 6.10 -- 6.12

Simultant assignment

$$x,y := e_1,e_2$$

- Simultant assignment $x,y := e_1,e_2$ evaluates e_1 and e_2 in the current state to values v_1 and v_2, then assign these to x and y respectively.

$$\mathbf{wp}\ (x,y := e_1,e_2)\ Q \ = \ Q[e_1,e_2/x,y]$$

- Simultant substitution $Q[e_1,e_2/x,y]$ replaces any free occurrence of x in Q with e_1, and y with e_2. In particular, we should **not** do the substitution of y **after** the substitution of x, nor the other way around.

The order of assignment matters

- The order in which you assign variables matters (you already know this). E.g. these three have different behavior:

wp $\boxed{(x:=x*y;\ y:=x+y)}$ $(x=2 \wedge y=1)$ = $\boxed{xy=2 \ \wedge\ xy+y=1}$

equivalent to: $x=-2 \wedge y=-1$

wp $\boxed{(y:=x+y;\ x:=x*y)}$ $(x=2 \wedge y=1)$ = $\boxed{x(x+y)=2 \wedge x+y=1}$

equivalent to: $x=2 \wedge y=-1$

wp $\boxed{(x,y := x*y,\ x+y)}$ $(x=2 \wedge y=1)$ = $\boxed{xy=2 \wedge x+y=1}$

which has no integer solution.

Assignment into an array

- Recall :

$$\textbf{wp}\ (x{:=}e)\ Q \quad = \quad Q[e/x]$$

- Unfortunately this rule does not extend to $a[e_1] := e_2$. Consider these examples, where the pre-condition is obtained by literally applying the above calculation rule:

(valid pre-cond) {* y = 10 *} a[i] := y {* a[i] = 10 *}

(not a valid pre-cond) {* a[0] = 10 *} a[i] := y {* a[0] = 10 *}

Fix : {* (i=0 → y | a[0]) = 10 *}

The formalism

- Let a be an array. Introduce this notation to mean the same array as a, except its i-th element, which is e:

$$a\ (i\ \textbf{repby}\ e)$$

- Formally defined via this equation:

$$a(i\ \textbf{repby}\ e)[k] \quad = \quad i=k \rightarrow e \mid a[k]$$

- Treat array assignments like a[i] := e as the assignment

$$a\ :=\ a(i\ \textbf{repby}\ e)$$

This has the same effect, but now we can safely use the old wp rule for assignment.

Example

- Prove: {* i≠k *} a[k]:= 0 ; a[i]:=1 {* a[k] = 0 *}
- Let's first calculate the **wp**:

{* i=k → 1 | 0 = 0 *} // def. repby

{* (1=k → 1 | a(k <u>repby</u> 0)[k]) = 0 *} // wp

a[k]:=0 // a := a(k repby 0)

{* (i=k → 1 | a[k]) = 0 *} // def. repby

{* a(i <u>repby</u> 1)[k] = 0 *} // wp

a[i]:=1 // a := a(i repby 1)

{* a[k] = 0 *}

- Now we need to prove that i≠k implies the wp.

Reducing a program spec to a statement spec

- Consider the program below with the given specification.

```
{* x > 0 *}   P(x:int) { x++ ; return x }   {*  return  = x+1 *}
```

Note: “return” in the post-condition is a special variable that refers to the P’s return value.

- Since we define our Hoare logic at the statement level, we first need to ”reduce” such a program-level specification to a specification in terms of its body.
- Such reduction raises several issues:
  - What is the logic of a ”return” statement ?
  - Should x in the poscond refer to its initial or final value?
  - How to refer to the “old” value of a variable?

Some restrictions imposed on uPL

- First, it is useful to be aware of some restrictions imposed on uPL which were introduced to simplify the resulting Hoare logic, allowing us to focus our study on its main concepts.
  - "**return**" statement should only appear as the last statement in the program.
  - Formulas in the pre- and post-conditions of a program-level specification of a program P can only refer to P's parameters or to "auxiliary variables" (explained later).
  - Parameters are either **passed by value**, or **passed by copy-restore**. In particular uPL does not allow references/pointers to be passed to a program.

Pass by copy-restore

- Arrays are by default passed by copy-restore. A parameter with “OUT” marker is passed by copy-restore. Example:

```
P(OUT  x, OUT y) { x := T ; y := F }
```

- The behavior of a call e.g. “P(a,b)” is as follows:

  The value of a and b are evaluated, and copied to P’s local variables x and y. The the body of P is executed. Then the final values of x and y are copied to a and b, in that order.

Pass by copy-restore, why?

- Why don't we just use pass-by-reference? Well, allowing this could create "**aliases**" (different variables pointing to the same data-cell). This complicate complicate the logic of assignment. Recall this logic:

$$\{*\ P\ *\}\ \ x{:=}e\ \ \{*\ Q\ *\} \quad = \quad P \Rightarrow Q[e/x]$$

  This assignment will now also affect the meaning of x's aliases! The above logic is too simplistic to handle this.
- In this course we want to avoid this complication, to focus more on the basic Hoare logic.
- Pass-by-copy-restore would still allow us to simulate programs with side effect, which is an interesting class to consider.

Meaning of variables in the post-condition

- If “y” is a pass-by-copy-restore parameter, in the post-condition it refers to its final value. Example:

{* true *} **P**(**OUT** y) { y := 1 } {* y>0 *}

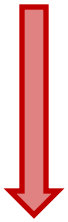
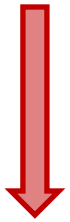
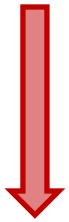
- If ”x” is a pass-by-value parameter, in the post-condition it refers to its **initial value.** Example:

{* true *} **P**(x, **OUT** y) { y := x ; x:=0 } {* y=x *}

Example of the reduction

- Consider again the previous example. To reduce the specification to the statement level we will do the following transformation:

{* x > 0 *} P(x:int) { x++ ; **return** x } {* return = x+1 *}

{* x > 0 *} **X**:=x ; x++ ; return := x {* return = **X**+1 *}

Introduce auxiliary variable(s) to remember the initial value of the parameters,

Replace pass-by-value parameters with reference to their initial values.

How to refer to parameters' old value?

- Consider a program P(OUT x) that increases the value of x by one. How to write a specification of this P? Again, we will use an auxiliary variable to remember x's old value:

```
{* true *}
   X:=x ; P(OUT x) { x := x+1 }
{* x = X+1 *}
```

Here, **X** is an **auxiliary variable**. It is not part of the program P. It is introduced just for the purpose of expressing P's specification.

```
{* true *}  X:=x ; x := x+1   {* x = X+1 *}
```

The corresponding statement-level specification.

Note: unstructured programs

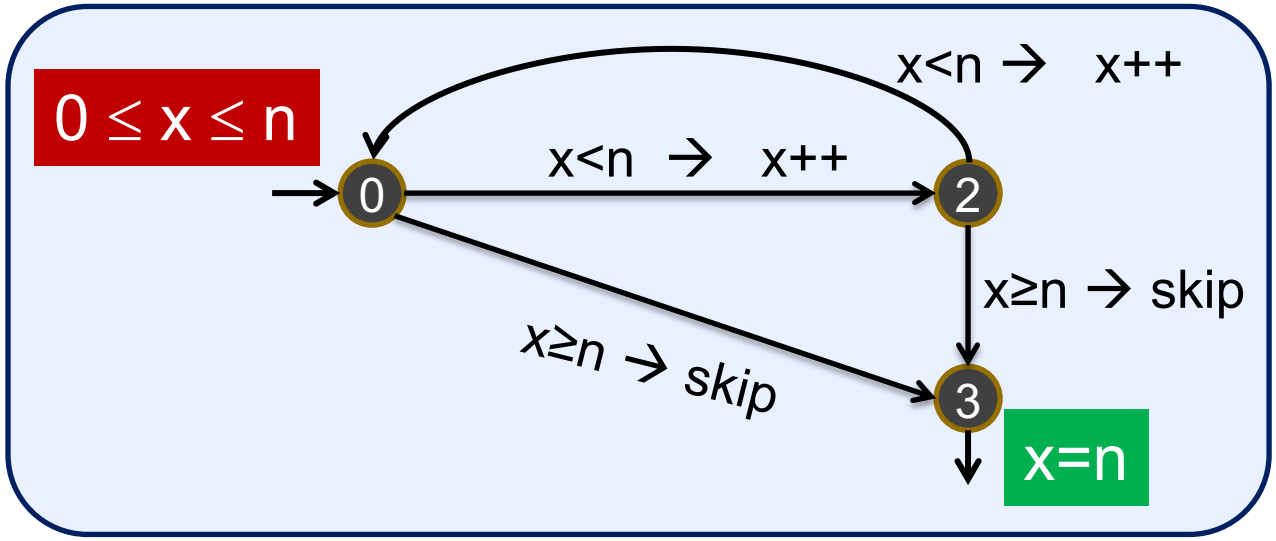
- uPL programs are "**structured**" programs. In such a program, the control flow follows the program's syntax.
- Most programs you write in an imperative language like Java or C# are structured.
- Unstructured programming on the other hand, allows arbitrary jump of control. Example:

```
start: if y=0 then goto exit ;
       if x>y then x:=x-y ; goto start
exit:  ...
```

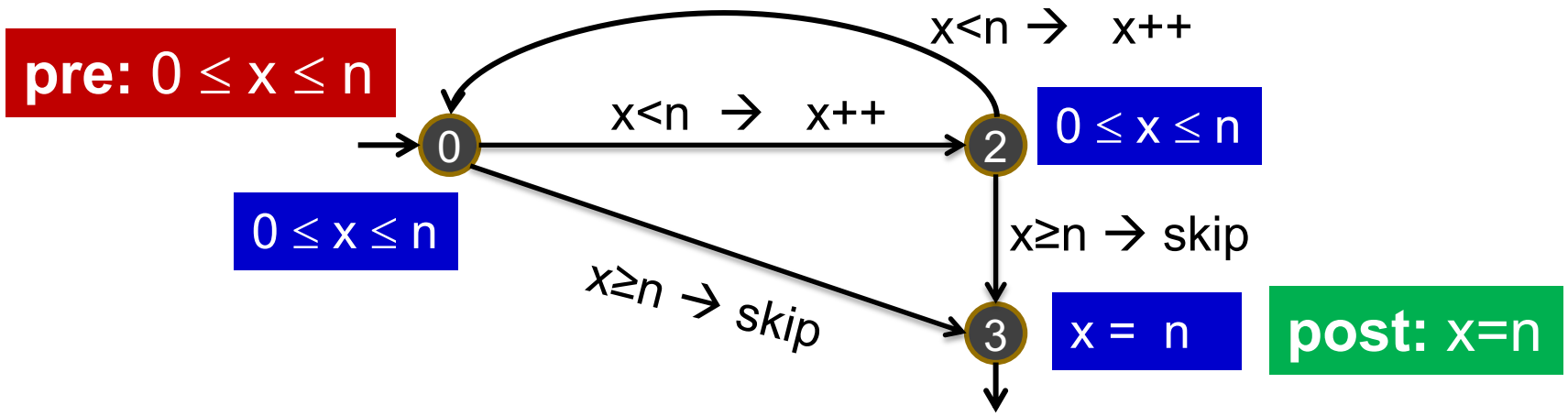
- Hoare logic does **not** work on unstructured programs. E.g. the rule for sequential composition would be broken!

Floyd Logic

Consider the following example of an unstructured program represented by a graph. The program start in node 0, and 3 is the exit node. Each arrow represent an assignment to execute, guarded by the corresponding condition.
The red and green boxes specify the pre- and post-condition of this program.



Floyd Logic



We decorate every node with an assertion (blue). Let A_s be the assertion at node s. Given a pre- and post-condition P and Q, they are valid if:

1. The assertions are consistent. That is: for every arrow $s \xrightarrow{a} t$, {* A_s*} *a* {* A_t *} holds.
2. P implies the assertion at the starting node.
3. The assertion at the exit node implies Q.
4. There is a termination metric m, with a lower bound 0. Executing any arrow decreases this m.